

DSA Revision Notes

Topic 1 — Arrays & Hashing

Complete theory · All operations · Complexity analysis
Patterns · Techniques · Solved examples · LeetCode prep

8

Core Sections

15+

Code Examples

12

Complexity Tables

25+

Key Patterns

Table of Contents

1. Introduction to Arrays

- Definition, Memory Layout, Types
- Static vs Dynamic Arrays

2. Array Operations & Complexity

- Access, Search, Insertion, Deletion
- Complete Big-O Table

3. Multi-dimensional Arrays

- 2D Arrays, Row/Column Major Order
- Matrix Traversal Patterns

4. Core Array Techniques

- Prefix Sum, Difference Array
- Two Pointers, Sliding Window, Kadane's

5. Sorting Algorithms

- Bubble, Selection, Insertion, Merge, Quick, Heap, Counting, Radix
- Full Complexity Comparison Table

6. Hashing — Core Concepts

- Hash Functions, Collision Handling
- Chaining vs Open Addressing

7. HashMap & HashSet Operations

- All Operations with Complexity
- Python dict / set internals

8. Array + Hashing Patterns

- Frequency Count, Running Sum, Index Mapping
- Two-Sum Family, Anagram Detection, Subarray Patterns

9. Common Interview Techniques

- Cyclic Sort, Dutch National Flag, Boyer-Moore Voting

10. Complexity Cheat Sheet

1. Introduction to Arrays

1.1 What is an Array?

An **array** is a linear data structure that stores a collection of elements of the **same data type** in **contiguous (adjacent) memory locations**. Each element is identified by an **index** (0-based in most languages). Because elements sit side-by-side in memory, random access is $O(1)$ — you compute the address arithmetically.

■ Address Formula

Address of element at index i = $\text{Base_Address} + i \times \text{sizeof}(\text{data_type})$

Example: `int arr[5]` at base 1000, `sizeof(int)=4` → `arr[3]` is at $1000 + 3 \times 4 = 1012$

1.2 Memory Layout

Arrays occupy a single contiguous block in memory. This locality makes them **cache-friendly** — accessing `arr[i]` and then `arr[i+1]` likely hits the same cache line, giving real-world speedups.

Index	0	1	2	3	4
Value	10	20	30	40	50
Address	1000	1004	1008	1012	1016

Figure 1: Memory layout of `int arr[5]` — each cell is 4 bytes apart

1.3 Static vs Dynamic Arrays

Property	Static Array	Dynamic Array (e.g. Python list)
Size	Fixed at compile time	Grows / shrinks at runtime
Memory	Stack (usually)	Heap
Access	$O(1)$	$O(1)$
Append	Not supported	Amortized $O(1)$
Insertion	$O(n)$ shift	$O(n)$ shift
Resize	Not possible	$O(n)$ copy to new buffer
Memory overhead	None	~1.5–2× capacity buffer
Examples	C <code>int[]</code> , Java <code>int[]</code>	Python list, Java <code>ArrayList</code> , C++ <code>vector</code>

1.4 Types of Arrays

- **1D Array**: Linear sequence. `arr = [1, 2, 3, 4, 5]`
- **2D Array**: Matrix / grid. `arr[row][col]`. Used in DP tables, image processing.
- **Jagged Array**: Array of arrays where rows can have different lengths.

- **Circular Array:** Treats index modulo n to simulate a ring buffer (deque internals).
- **Sorted Array:** Maintained in sorted order. Enables binary search in $O(\log n)$.
- **Bit Array:** Each element is a single bit. Ultra-compact. Used in Bloom Filters.

2. Array Operations & Complexity

2.1 Access (Read / Write)

Accessing an element by index is the **fastest array operation**: $O(1)$ time and $O(1)$ space. The CPU computes the physical address in one step using the base + offset formula.

```
Python
1 arr = [10, 20, 30, 40, 50]
2
3 # Read
4 val = arr[2]          #  $O(1) \rightarrow \text{val} = 30$ 
5
6 # Write
7 arr[2] = 99           #  $O(1) \rightarrow \text{arr} = [10, 20, 99, 40, 50]$ 
8
9 # Negative index (Python)
10 last = arr[-1]       #  $O(1) \rightarrow \text{last} = 50$ 
```

2.2 Search

Linear Search — $O(n)$

Scan every element until target is found or the array ends. No ordering required. Best case $O(1)$ (found at index 0), worst case $O(n)$ (not present).

```
Python
1 def linear_search(arr, target):
2     for i, val in enumerate(arr):
3         if val == target:
4             return i          # found at index i
5     return -1                 # not found
6
7 # Time:  $O(n)$    Space:  $O(1)$ 
```

Binary Search — $O(\log n)$

Prerequisite: array must be sorted. Repeatedly halve the search space. After k steps, search space = $n / 2^k$. When $n / 2^k = 1 \rightarrow k = \log_2(n)$. Three templates exist: exact match, lower-bound, upper-bound.

Python

```
1 def binary_search(arr, target):
2     lo, hi = 0, len(arr) - 1
3     while lo <= hi:
4         mid = lo + (hi - lo) // 2 # avoids overflow
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             lo = mid + 1
9         else:
10            hi = mid - 1
11    return -1
12
13 # Time: O(log n)   Space: O(1)
```

■ Binary Search Variants

Lower Bound: first index where $\text{arr}[i] \geq \text{target}$ → while $\text{lo} < \text{hi}$: if $\text{arr}[\text{mid}] < \text{target}$: $\text{lo} = \text{mid} + 1$ else: $\text{hi} = \text{mid}$

Upper Bound: first index where $\text{arr}[i] > \text{target}$ → while $\text{lo} < \text{hi}$: if $\text{arr}[\text{mid}] \leq \text{target}$: $\text{lo} = \text{mid} + 1$ else: $\text{hi} = \text{mid}$

Safe mid: always use $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) // 2$ to prevent integer overflow in Java/C++

2.3 Insertion

Inserting at the **end** of a dynamic array is amortized $O(1)$ — most of the time it's just placing a value, and occasionally the buffer doubles ($O(n)$ copy spread over n inserts → $O(1)$ amortized). Inserting at an arbitrary index i requires shifting $n-i$ elements right → $O(n)$.

Python

```
1 # Insert at end: O(1) amortized
2 arr.append(val)
3
4 # Insert at index i: O(n)
5 arr.insert(i, val) # shifts arr[i..n-1] one step right
6
7 # Manual shift (to understand internals):
8 def insert_at(arr, i, val):
9     arr.append(None) # grow by 1
10    for j in range(len(arr)-1, i, -1):
11        arr[j] = arr[j-1] # shift right
12    arr[i] = val
13    # Time: O(n)   Space: O(1)
```

2.4 Deletion

Deleting the **last element** is $O(1)$. Deleting at index i shifts $n-i-1$ elements left $\rightarrow O(n)$. A common trick for $O(1)$ deletion (when order doesn't matter): swap target with last element, then pop.

Python

```

1 # Delete last element: O(1)
2 arr.pop()
3
4 # Delete at index i: O(n)
5 arr.pop(i)          # shifts arr[i+1..n-1] left
6
7 # O(1) deletion trick (order doesn't matter):
8 def delete_01(arr, i):
9     arr[i] = arr[-1] # overwrite with last element
10    arr.pop()        # remove last
11    # Time: O(1)   Space: O(1) – but order changes!

```

2.5 Complete Complexity Table

Operation	Best	Average	Worst	Space	Notes
Access by index	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Direct address calc
Search (unsorted)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Linear scan
Search (sorted)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Binary search
Insert at end	$O(1)$	$O(1)^*$	$O(n)$	$O(1)$	*Amortized; resize is $O(n)$
Insert at index i	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Shift $n-i$ elements
Insert at beginning	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Shift all elements
Delete last	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Just decrement size
Delete at index i	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Shift $n-i-1$ elements
Delete (swap with last)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Only if order irrelevant
Traverse all elements	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Single pass
Find min / max	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Must check all
Reverse	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Two-pointer swap
Rotate by k	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Reverse 3x trick
Sort (comparison)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Depends on algo
Copy array	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Full traversal copy

Table 1: Complete Array Operations Complexity

3. Multi-dimensional Arrays

3.1 2D Arrays (Matrices)

A 2D array is an array of arrays. Element at row r , column c is accessed as **arr[r][c]**. In row-major order (C, Python, Java), row r lives at consecutive addresses: $\text{Address} = \text{Base} + (r \times \text{cols} + c) \times \text{sizeof}(\text{type})$.

Python

```
1 # Create 3x4 matrix
2 matrix = [[0] * 4 for _ in range(3)] # 3 rows, 4 cols
3
4 # Access: 0(1)
5 val = matrix[1][2] # row 1, col 2
6
7 # Row traversal: 0(cols)
8 for col in range(4):
9     print(matrix[0][col])
10
11 # Full traversal: 0(rows * cols)
12 for r in range(3):
13     for c in range(4):
14         print(matrix[r][c])
```

3.2 Matrix Traversal Patterns

- **Row-by-Row:** for r in range(rows): for c in range(cols) — Cache-friendly (row-major)
- **Column-by-Column:** for c in range(cols): for r in range(rows) — Cache-unfriendly
- **Diagonal:** for d in range(rows): cell = matrix[d][d] — Main diagonal
- **Spiral:** Use 4 boundary pointers; shrink inward — LC 54 — Spiral Matrix
- **Transpose:** swap(matrix[r][c], matrix[c][r]) for $r < c$ — In-place for square matrix
- **Rotate 90° CW:** Transpose then reverse each row — In-place $O(n^2)$
- **Anti-diagonal:** for d in range(rows+cols-1): vary r, c by d — Sum diagonals pattern
- **BFS/DFS on grid:** Treat each cell as graph node with 4 neighbors — LC 200 — Number of Islands

■ Grid Traversal Template

Direction arrays for 4-directional grid BFS/DFS:

dr = [-1, 1, 0, 0] dc = [0, 0, -1, 1]

For 8-directional: dr = [-1,-1,-1,0,0,1,1,1] dc = [-1,0,1,-1,1,-1,0,1]

Always check: $0 \leq nr < \text{rows}$ and $0 \leq nc < \text{cols}$ before visiting

4. Core Array Techniques

4.1 Prefix Sum

Precompute cumulative sums so any **subarray sum** query is answered in $O(1)$ instead of $O(n)$. Trade-off: $O(n)$ preprocessing + $O(n)$ space. Core identity: $\text{sum}(l..r) = \text{prefix}[r+1] - \text{prefix}[l]$

Python

```
1 def build_prefix(arr):
2     n = len(arr)
3     prefix = [0] * (n + 1)      # prefix[0] = 0
4     for i in range(n):
5         prefix[i+1] = prefix[i] + arr[i]
6     return prefix
7
8 def range_sum(prefix, l, r):    # inclusive [l, r]
9     return prefix[r+1] - prefix[l] # O(1) query
10
11 # Example: arr = [3, 1, 4, 1, 5]
12 # prefix = [0, 3, 4, 8, 9, 14]
13 # sum(1..3) = prefix[4] - prefix[1] = 9 - 3 = 6 ✓
14
15 # Time: O(n) build + O(1) query   Space: O(n)
```

■ 2D Prefix Sum

2D Prefix Sum: $\text{prefix}[r][c] = \text{arr}[r][c] + \text{prefix}[r-1][c] + \text{prefix}[r][c-1] - \text{prefix}[r-1][c-1]$

Rectangle sum (r1,c1) to (r2,c2) = $\text{prefix}[r2][c2] - \text{prefix}[r1-1][c2] - \text{prefix}[r2][c1-1] + \text{prefix}[r1-1][c1-1]$

Use case: LC 304 Range Sum Query 2D, LC 560 Subarray Sum Equals K (running sum + hashmap)

4.2 Difference Array

Apply **range updates** in $O(1)$ and reconstruct the modified array in $O(n)$. $\text{diff}[i] = \text{arr}[i] - \text{arr}[i-1]$. To add val to $\text{arr}[l..r]$: $\text{diff}[l] += \text{val}$; $\text{diff}[r+1] -= \text{val}$.

Python

```
1 # Range update in O(1) using difference array
2 n = 5
3 diff = [0] * (n + 1)
4
5 def range_update(l, r, val):
6     diff[l] += val
7     if r + 1 <= n:
8         diff[r + 1] -= val
9
10 def build_result():
11     result = []
12     running = 0
13     for d in diff[:n]:
14         running += d
15         result.append(running)
16     return result
17
18 # Apply updates, then build once: O(n)
```

4.3 Two Pointers

Use two indices (left and right) that move toward each other or in the same direction. Converts $O(n^2)$ brute-force solutions to $O(n)$. Works best on **sorted arrays** or when processing from both ends.

Python

```
1 # Pattern 1: Opposite ends (sorted array) – Two Sum
2 def two_sum_sorted(arr, target):
3     l, r = 0, len(arr) - 1
4     while l < r:
5         s = arr[l] + arr[r]
6         if s == target: return (l, r)
7         elif s < target: l += 1
8         else: r -= 1
9     return (-1, -1) # Time: O(n) Space: O(1)
10
11 # Pattern 2: Same direction – Remove Duplicates
12 def remove_duplicates(arr):
13     slow = 0
14     for fast in range(1, len(arr)):
15         if arr[fast] != arr[slow]:
16             slow += 1
17             arr[slow] = arr[fast]
18     return slow + 1 # new length
```

4.4 Sliding Window

Maintain a window $[l, r]$ and expand/shrink it based on a condition. **Fixed window**: both pointers move together. **Variable window**: expand right, shrink left when invalid. Avoids recomputing the window from scratch every step.

Python

```
1 # Fixed window – max sum of size k
2 def max_sum_k(arr, k):
3     window = sum(arr[:k])
4     best = window
5     for i in range(k, len(arr)):
6         window += arr[i] - arr[i - k] # slide
7         best = max(best, window)
8     return best # Time: O(n) Space: O(1)
9
10 # Variable window – longest substring without repeat
11 def length_of_longest(s):
12     seen, l, best = {}, 0, 0
13     for r, ch in enumerate(s):
14         if ch in seen and seen[ch] >= l:
15             l = seen[ch] + 1 # shrink
16         seen[ch] = r
17         best = max(best, r - l + 1)
18     return best # Time: O(n) Space: O(k)
```

4.5 Kadane's Algorithm (Maximum Subarray)

Find the contiguous subarray with the **largest sum** in $O(n)$ time. Key insight: if the running sum ever becomes negative, discard it and start fresh. Local max at each step: $\max(\text{arr}[i], \text{local_max} + \text{arr}[i])$.

Python

```
1 def max_subarray(arr):
2     local_max = global_max = arr[0]
3     for i in range(1, len(arr)):
4         local_max = max(arr[i], local_max + arr[i])
5         global_max = max(global_max, local_max)
6     return global_max
7
8 # Time: O(n) Space: O(1)
9 # Extension: track indices by storing start/end when updating global_max
10 # Variant: LC 152 – Maximum Product Subarray (track both min and max)
```

■ Kadane's Variants

Circular Subarray: max of (Kadane on arr) vs (total_sum - Kadane on -arr)

Max Product: maintain max_prod and min_prod (negatives flip signs)

At least K elements: use prefix min DP variant

5. Sorting Algorithms

Bubble Sort

Property	Value
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$
Stable?	Stable

- Repeatedly swap adjacent elements if out of order.
- After each pass, the largest unsorted element bubbles to its correct position.
- Optimise: stop early if no swaps in a pass (already sorted).

Python

```
1 def bubble_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         swapped = False
5         for j in range(n - i - 1):
6             if arr[j] > arr[j+1]:
7                 arr[j], arr[j+1] = arr[j+1], arr[j]
8                 swapped = True
9         if not swapped: break # early exit
```

Selection Sort

Property	Value
Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$
Stable?	Not Stable

- Find the minimum element in the unsorted portion and swap it to the front.
- Always $O(n^2)$ comparisons regardless of input — no early exit possible.
- Minimises swaps: $O(n)$ swaps total (useful if swaps are costly).

Python

```
1 def selection_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         min_idx = i
5         for j in range(i+1, n):
6             if arr[j] < arr[min_idx]:
7                 min_idx = j
8         arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Insertion Sort

Property	Value
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$
Stable?	Stable

- Build sorted portion one element at a time by inserting into correct position.
- Best case $O(n)$ for already-sorted input — excellent for small or nearly sorted arrays.
- Used as base case in hybrid sorts (Timsort, Introsort) for small n .

Python

```
1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key = arr[i]
4         j = i - 1
5         while j >= 0 and arr[j] > key:
6             arr[j+1] = arr[j]
7             j -= 1
8         arr[j+1] = key
```

Merge Sort

Property	Value
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$
Space	$O(n)$
Stable?	Stable

- Divide array in half recursively, sort each half, merge sorted halves.
- Guaranteed $O(n \log n)$ — no worst-case degradation unlike quicksort.
- Used for linked lists (no random access needed) and external sorting.

Python

```

1 def merge_sort(arr):
2     if len(arr) <= 1: return arr
3     mid = len(arr) // 2
4     L = merge_sort(arr[:mid])
5     R = merge_sort(arr[mid:])
6     return merge(L, R)
7
8 def merge(L, R):
9     res, i, j = [], 0, 0
10    while i < len(L) and j < len(R):
11        if L[i] <= R[j]: res.append(L[i]); i += 1
12        else:           res.append(R[j]); j += 1
13    return res + L[i:] + R[j:]

```

Quick Sort

Property	Value
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$
Space	$O(\log n)$
Stable?	Not Stable

- Choose a pivot; partition: all $<$ pivot go left, all $>$ go right; recurse.
- Worst case $O(n^2)$ when pivot is always min/max — use random pivot to avoid.
- Average-case fastest in practice due to cache efficiency and small constant.

Python

```
1 import random
2 def quick_sort(arr, l=0, r=None):
3     if r is None: r = len(arr) - 1
4     if l >= r: return
5     pivot_idx = random.randint(l, r)
6     arr[pivot_idx], arr[r] = arr[r], arr[pivot_idx]
7     p = partition(arr, l, r)
8     quick_sort(arr, l, p - 1)
9     quick_sort(arr, p + 1, r)
10
11 def partition(arr, l, r):
12     pivot = arr[r]; i = l - 1
13     for j in range(l, r):
14         if arr[j] <= pivot:
15             i += 1
16             arr[i], arr[j] = arr[j], arr[i]
17     arr[i+1], arr[r] = arr[r], arr[i+1]
18     return i + 1
```

Heap Sort

Property	Value
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$
Space	$O(1)$
Stable?	Not Stable

- Build a max-heap from the array in $O(n)$, then extract max n times.
- Each extraction takes $O(\log n)$, total $O(n \log n)$. In-place and guaranteed $O(n \log n)$.
- Not cache-friendly; slower than quicksort in practice despite same asymptotic.

Python

```

1 def heap_sort(arr):
2     n = len(arr)
3     # Build max-heap
4     for i in range(n // 2 - 1, -1, -1):
5         heapify(arr, n, i)
6     # Extract elements
7     for i in range(n-1, 0, -1):
8         arr[0], arr[i] = arr[i], arr[0]
9         heapify(arr, i, 0)
10
11 def heapify(arr, n, i):
12     largest = i; l = 2*i+1; r = 2*i+2
13     if l < n and arr[l] > arr[largest]: largest = l
14     if r < n and arr[r] > arr[largest]: largest = r
15     if largest != i:
16         arr[i], arr[largest] = arr[largest], arr[i]
17         heapify(arr, n, largest)

```

Counting Sort

Property	Value
Best	$O(n+k)$
Average	$O(n+k)$
Worst	$O(n+k)$
Space	$O(k)$
Stable?	Stable

- Non-comparison sort. Count occurrences, then rebuild. k = range of values.
- Only works for integers (or mappable to integers) in a known range.
- $O(n+k)$ — faster than $O(n \log n)$ when $k = O(n)$. Used in radix sort.

Python

```

1 def counting_sort(arr):
2     if not arr: return arr
3     min_v, max_v = min(arr), max(arr)
4     count = [0] * (max_v - min_v + 1)
5     for x in arr:
6         count[x - min_v] += 1
7     result = []
8     for i, c in enumerate(count):
9         result.extend([i + min_v] * c)
10    return result    # Time  $O(n+k)$  Space  $O(k)$ 

```

Radix Sort

Property	Value
Best	$O(d \cdot n)$
Average	$O(d \cdot n)$
Worst	$O(d \cdot n)$
Space	$O(n+k)$
Stable?	Stable

- Sort digit by digit from least significant to most significant.
- d = number of digits, k = digit range (10 for decimal). Efficient when d is small.
- Outperforms comparison sorts when $d \ll \log n$ (e.g., sorting 32-bit integers).

Python

```
1 def radix_sort(arr):
2     max_v = max(arr)
3     exp = 1
4     while max_v // exp > 0:
5         counting_sort_by_digit(arr, exp)
6         exp *= 10
7
8 def counting_sort_by_digit(arr, exp):
9     n = len(arr); output = [0]*n; count = [0]*10
10    for num in arr: count[(num//exp)%10] += 1
11    for i in range(1,10): count[i] += count[i-1]
12    for i in range(n-1,-1,-1):
13        d = (arr[i]//exp)%10
14        output[count[d]-1] = arr[i]; count[d] -= 1
15    for i in range(n): arr[i] = output[i]
```

Sorting Algorithm Comparison

Algorithm	Best	Average	Worst	Space	Stable	Use When
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Never in prod; teaching only
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Minimise swaps
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Small / nearly sorted arrays
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Linked list, stable needed
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	General; fastest in practice
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Guaranteed $O(n \log n)$ in-place
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes	Small integer range
Radix Sort	$O(dn)$	$O(dn)$	$O(dn)$	$O(n+k)$	Yes	Fixed-size integers / strings
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Python's built-in sort

Table 2: Sorting Algorithm Comparison

6. Hashing — Core Concepts

6.1 What is a Hash Function?

A **hash function** maps a key of arbitrary size to an integer in a fixed range $[0, m-1]$ where m is the table size. A good hash function is: **deterministic**, **uniform** (spreads keys evenly), **fast** to compute $O(1)$, and **minimises collisions**.

Python

```
1 # Division method:  $h(k) = k \bmod m$ 
2 # Best when  $m$  is a prime not near a power of 2
3
4 # Multiplication method:  $h(k) = \text{floor}(m * \text{frac}(k * A))$ 
5 #  $A = (\text{sqrt}(5)-1)/2 \approx 0.618$  (Knuth's suggestion)
6
7 # String hashing (polynomial rolling):
8 def poly_hash(s, base=31, mod=10**9+7):
9     h = 0
10    for ch in s:
11        h = (h * base + ord(ch) - ord('a') + 1) % mod
12    return h
13
14 # Python's built-in hash() uses SipHash-1-3 (randomised per process)
```

6.2 Collision Handling

Chaining (Separate Chaining)

Each bucket holds a linked list (or dynamic array) of all keys mapping to that slot. Search, insert, delete all $O(1+\alpha)$ average where $\alpha = n/m$ is the **load factor**. Python's dict uses open addressing (not chaining).

Python

```

1 # Chaining example (conceptual)
2 class HashMap:
3     def __init__(self, size=1000):
4         self.size = size
5         self.buckets = [[] for _ in range(size)]
6
7     def _hash(self, key):
8         return hash(key) % self.size
9
10    def put(self, key, val):
11        idx = self._hash(key)
12        for i, (k, v) in enumerate(self.buckets[idx]):
13            if k == key:
14                self.buckets[idx][i] = (key, val) # update
15                return
16        self.buckets[idx].append((key, val)) # insert
17
18    def get(self, key): # O(1) avg
19        idx = self._hash(key)
20        for k, v in self.buckets[idx]:
21            if k == key: return v
22        return -1

```

Open Addressing

All entries stored in the hash table itself. On collision, probe for the next open slot.

- **Linear Probing:** $h(k, i) = (h(k) + i) \bmod m$ — Simple but causes primary clustering
- **Quadratic Probing:** $h(k, i) = (h(k) + i^2) \bmod m$ — Reduces primary clustering; secondary clustering possible
- **Double Hashing:** $h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$ — Best distribution; almost no clustering

6.3 Chaining vs Open Addressing

Property	Chaining	Open Addressing
Space	Extra node memory	No extra memory
Load factor (α)	Can exceed 1.0	Must stay < 1.0
Search (avg)	$O(1 + \alpha)$	$O(1/(1-\alpha))$
Cache performance	Poor (pointer chasing)	Better (contiguous)
Deletion	Simple (remove from list)	Needs tombstone markers
Resize trigger	$\alpha > 0.75$ typically	$\alpha > 0.5-0.7$ typically
Real usage	Java HashMap	Python dict, C++ unordered_map

7. HashMap & HashSet Operations

7.1 Python dict — All Operations

[illegible]

Operation	Average	Worst	Notes
Insert (put)	$O(1)$	$O(n)$	Worst when all keys collide
Lookup (get)	$O(1)$	$O(n)$	n = bucket chain length
Delete	$O(1)$	$O(n)$	Find then remove
Membership test	$O(1)$	$O(n)$	Same as lookup
Iterate all keys	$O(n)$	$O(n)$	Visit every bucket
Set union	$O(n+m)$	$O(n+m)$	n, m = set sizes
Set intersection	$O(\min)$	$O(\min)$	Iterate smaller set
Resize / rehash	$O(n)$	$O(n)$	Amortized $O(1)$ per insert
Space	$O(n)$	$O(n)$	Plus overhead (~2x bucket array)

Table 3: *HashMap / HashSet Operations Complexity*

8. Array + Hashing Patterns

Pattern 1: Frequency Count

Count how many times each element appears. Enables $O(1)$ lookup for 'has element been seen?' and 'how many times?'. Foundation for anagram detection, top-K, and majority element problems.

Python

```
1 from collections import Counter, defaultdict
2
3 # LC 242 - Valid Anagram
4 def is_anagram(s: str, t: str) -> bool:
5     return Counter(s) == Counter(t) # O(n) time, O(1) space (26 letters)
6
7 # LC 347 - Top K Frequent Elements
8 def top_k_frequent(nums, k):
9     count = Counter(nums)
10    return [x for x, _ in count.most_common(k)] # O(n log k)
11
12 # LC 169 - Majority Element (Boyer-Moore + freq map approach)
13 def majority_element(nums):
14     count = Counter(nums)
15     return max(count, key=count.get) # O(n) time, O(n) space
16
17 # O(1) space majority: Boyer-Moore Voting
18 def majority_bm(nums):
19     cand, cnt = nums[0], 1
20     for x in nums[1:]:
21         cnt += (1 if x == cand else -1)
22         if cnt == 0: cand, cnt = x, 1
23     return cand
```

Pattern 2: Two-Sum Family (Index Mapping)

Store a value $\rightarrow$ index mapping to find pairs, triples, or complements in $O(n)$ instead of $O(n^2)$. Core idea: for each x , check if $(\text{target} - x)$ is already in the map.

Python

```
1 # LC 1 – Two Sum
2 def two_sum(nums, target):
3     seen = {}          # value → index
4     for i, x in enumerate(nums):
5         complement = target - x
6         if complement in seen:
7             return [seen[complement], i]
8         seen[x] = i
9     return []          # Time: O(n) Space: O(n)
10
11 # LC 167 – Two Sum II (sorted) – O(n) time, O(1) space
12 def two_sum_sorted(nums, target):
13     l, r = 0, len(nums) - 1
14     while l < r:
15         s = nums[l] + nums[r]
16         if s == target: return [l+1, r+1]
17         elif s < target: l += 1
18         else: r -= 1
19
20 # LC 15 – 3Sum – fix one, two-pointer on rest
21 def three_sum(nums):
22     nums.sort(); res = []
23     for i, x in enumerate(nums):
24         if i > 0 and x == nums[i-1]: continue # skip duplicates
25         l, r = i+1, len(nums)-1
26         while l < r:
27             s = x + nums[l] + nums[r]
28             if s == 0: res.append([x, nums[l], nums[r]]); l += 1; r -= 1
29             elif s < 0: l += 1
30             else: r -= 1
31     return res          # Time: O(n^2) Space: O(n)
```

Pattern 3: Running Sum + HashMap (Subarray Sum)

Combine prefix sums with a hashmap to find subarrays with a given property in $O(n)$. Core insight: $\text{sum}(l..r) = \text{prefix}[r] - \text{prefix}[l-1]$. So $\text{prefix}[r] - k = \text{prefix}[l-1]$. Store prefix sums in a map and look up the required complement.

Python

```
1 # LC 560 – Subarray Sum Equals K
2 def subarray_sum(nums, k):
3     prefix_count = {0: 1}    # prefix_sum → frequency
4     running, count = 0, 0
5     for x in nums:
6         running += x
7         # Need (running - k) to have occurred before
8         count += prefix_count.get(running - k, 0)
9         prefix_count[running] = prefix_count.get(running, 0) + 1
10    return count    # Time: O(n) Space: O(n)
11
12 # LC 525 – Contiguous Array (equal 0s and 1s)
13 def find_max_length(nums):
14     # Convert 0 → -1; find longest subarray with sum 0
15     first_seen = {0: -1}
16     running, best = 0, 0
17     for i, x in enumerate(nums):
18         running += 1 if x == 1 else -1
19         if running in first_seen:
20             best = max(best, i - first_seen[running])
21         else:
22             first_seen[running] = i
23     return best
```

Pattern 4: Group Anagrams / Canonical Form

Python

```
1 # LC 49 – Group Anagrams
2 from collections import defaultdict
3
4 def group_anagrams(strs):
5     groups = defaultdict(list)
6     for s in strs:
7         key = tuple(sorted(s)) # canonical form: sorted tuple
8         groups[key].append(s)
9     return list(groups.values()) # Time: O(n·k·log k)
10
11 # O(n·k) version using character count as key:
12 def group_anagrams_fast(strs):
13     groups = defaultdict(list)
14     for s in strs:
15         cnt = [0] * 26
16         for ch in s: cnt[ord(ch) - ord('a')] += 1
17         groups[tuple(cnt)].append(s) # tuple is hashable
18     return list(groups.values())
```

Pattern 5: Longest Consecutive Sequence

Python

```
1 # LC 128 – Longest Consecutive Sequence – O(n)
2 def longest_consecutive(nums):
3     num_set = set(nums) # O(1) lookup
4     best = 0
5     for x in num_set:
6         if x - 1 not in num_set: # x is a sequence START
7             length = 1
8             while x + length in num_set:
9                 length += 1
10            best = max(best, length)
11    return best
12 # Each number is visited at most twice → O(n) total
```

9. Interview Techniques

9.1 Cyclic Sort

Used when array contains numbers in range $[1, n]$ or $[0, n]$. Place each number at its correct index: $arr[i]$ should be $i+1$ (or i). One pass: $O(n)$ time, $O(1)$ space. Use for finding missing/duplicate numbers.

Python

```
1 # Template: nums in range [1, n]
2 def cyclic_sort(nums):
3     i = 0
4     while i < len(nums):
5         correct = nums[i] - 1      # correct index for nums[i]
6         if nums[i] != nums[correct]: # not in right place
7             nums[i], nums[correct] = nums[correct], nums[i]
8         else:
9             i += 1
10    return nums
11
12 # LC 268 — Missing Number
13 def missing_number(nums):
14     i = 0
15     while i < len(nums):
16         j = nums[i]
17         if j < len(nums) and nums[j] != nums[i]:
18             nums[i], nums[j] = nums[j], nums[i]
19         else: i += 1
20     for i, x in enumerate(nums):
21         if x != i: return i
22     return len(nums)
```

9.2 Dutch National Flag (3-Way Partition)

Sort an array with 3 distinct values in $O(n)$ time and $O(1)$ space. Uses three pointers: lo (left boundary of 0s), mid (current), hi (right boundary of 2s). Foundation for 3-way quicksort partition.

Python

```
1 # LC 75 – Sort Colors (0, 1, 2)
2 def sort_colors(nums):
3     lo, mid, hi = 0, 0, len(nums) - 1
4     while mid <= hi:
5         if nums[mid] == 0:
6             nums[lo], nums[mid] = nums[mid], nums[lo]
7             lo += 1; mid += 1
8         elif nums[mid] == 1:
9             mid += 1
10        else: # nums[mid] == 2
11            nums[mid], nums[hi] = nums[hi], nums[mid]
12            hi -= 1
13            # DO NOT increment mid (swapped value unverified)
14    # Invariant: [0..lo-1]=0, [lo..mid-1]=1, [hi+1..n-1]=2
```

9.3 Boyer-Moore Voting Algorithm

Find the majority element (appears $> n/2$ times) in $O(n)$ time and $O(1)$ space. Key insight: pair up different elements and cancel them; the survivor is the majority. Works because majority element has more occurrences than all others combined.

Python

```
1 # LC 169 – Majority Element
2 def majority_element(nums):
3     candidate, count = nums[0], 1
4     for x in nums[1:]:
5         if count == 0:
6             candidate, count = x, 1
7         elif x == candidate:
8             count += 1
9         else:
10            count -= 1
11    return candidate # O(n) time, O(1) space
12
13 # LC 229 – Majority Element II (> n/3 times → at most 2 candidates)
14 def majority_element_ii(nums):
15     cand1, cand2, cnt1, cnt2 = None, None, 0, 0
16     for x in nums:
17         if x == cand1: cnt1 += 1
18         elif x == cand2: cnt2 += 1
19         elif cnt1 == 0: cand1, cnt1 = x, 1
20         elif cnt2 == 0: cand2, cnt2 = x, 1
21         else: cnt1 -= 1; cnt2 -= 1
22    return [c for c in [cand1, cand2] if nums.count(c) > len(nums)//3]
```

9.4 Array Rotation

```
Python

1 # LC 189 – Rotate Array by k – O(n) time, O(1) space
2 # Key trick: reverse three segments
3 def rotate(nums, k):
4     n = len(nums)
5     k %= n
6     reverse(nums, 0, n-1)    # step 1: reverse all
7     reverse(nums, 0, k-1)    # step 2: reverse first k
8     reverse(nums, k, n-1)    # step 3: reverse rest
9
10 def reverse(nums, l, r):
11     while l < r:
12         nums[l], nums[r] = nums[r], nums[l]
13         l += 1; r -= 1
14
15 # Example: [1,2,3,4,5,6,7] k=3
16 # After step 1: [7,6,5,4,3,2,1]
17 # After step 2: [5,6,7,4,3,2,1]
18 # After step 3: [5,6,7,1,2,3,4] ✓
```

10. Complexity Cheat Sheet

Big-O Quick Reference

Notation	Name	Example	n=1000
$O(1)$	Constant	Array access, hashmap get	1
$O(\log n)$	Logarithmic	Binary search	10
$O(n)$	Linear	Linear scan, prefix sum	1,000
$O(n \log n)$	Linearithmic	Merge sort, heap sort	10,000
$O(n^2)$	Quadratic	Bubble sort, naive 2-sum	1,000,000
$O(n^3)$	Cubic	Matrix chain mult	1,000,000,000
$O(2^n)$	Exponential	Subsets, brute backtrack	2^{1000} (unusable)
$O(n!)$	Factorial	All permutations	$n!$ (unusable)

Arrays & Hashing — Final Quick Reference

Need to...	Use...	Complexity
Access element by index	<code>arr[i]</code>	$O(1)$
Search in unsorted array	Linear scan	$O(n)$
Search in sorted array	Binary search	$O(\log n)$
Insert at end	<code>arr.append(x)</code>	$O(1)^*$
Insert at position i	<code>arr.insert(i, x)</code>	$O(n)$
Delete last	<code>arr.pop()</code>	$O(1)$
Delete at position i	<code>arr.pop(i)</code>	$O(n)$
Find subarray sum [l, r]	Prefix sum	$O(1)$ query
Range update on many indices	Difference array	$O(1)$ update
Max sum contiguous subarray	Kadane's algorithm	$O(n)$
Find if pair sums to target	Two sum + hashmap	$O(n)$
Count element frequencies	Counter / dict	$O(n)$
Group by canonical form	Sort key hashmap	$O(n \cdot k \log k)$
Subarray with sum K	Running sum + hashmap	$O(n)$
Sort n integers in [0, k]	Counting sort	$O(n+k)$
Sort n numbers general	Merge/Heap sort	$O(n \log n)$

Majority element	Boyer-Moore voting	$O(n)$, $O(1)$
Rotate array by k	3× reverse trick	$O(n)$, $O(1)$
Partition into 3 groups	Dutch National Flag	$O(n)$, $O(1)$
Find missing number $[1..n]$	Cyclic sort	$O(n)$, $O(1)$
Longest consecutive sequence	HashSet + anchor	$O(n)$

■ Golden Rules for Arrays & Hashing

1. Default to hashmap when you need $O(1)$ lookup for 'seen before' or 'complement exists'.
2. Prefix sum + hashmap solves most subarray property problems in $O(n)$.
3. Two pointers on sorted arrays / same-direction fast-slow = $O(n)$, no extra space.
4. When array has n elements in range $[1..n]$, think cyclic sort or XOR/math tricks.
5. For top-K, use a heap of size K : $O(n \log K)$. For all frequencies, use Counter.
6. Always check: can I sort first and apply binary search / two pointers?
7. Sliding window variable size: expand right freely, shrink left when window invalid.